

PHASE 0 — SMARTSAFE PROJECT ENVIRONMENT SETUP (Windows — Guaranteed to work)

Purpose: create a repeatable, portable conda environment and folder structure so the SmartSafe code and notebooks run on any Windows machine.

Important portability notes

- Use the %USERPROFILE% environment variable (Windows) or a relative path inside the repo to avoid embedding any personal username.
 - All commands below assume you have Anaconda / Miniconda installed and are run on **Windows**.
 - This document is the single, canonical Phase 0 — follow the steps exactly in order.
-

STEP 0 — Project folder (create once)

Open File Explorer and create a project folder (example using environment variable so it is portable):

%USERPROFILE%\smartsafe_project

Inside this folder place:

- cognitive_package_v2/ (folder)
- cognitive_model.joblib (file)
- terrain_data/ (folder with any GIS raster/vector inputs)
- All Jupyter notebooks for the project (*.ipynb)
- smartsafe_env.yml (we create next)

(If you prefer a repo-relative path, use ./smartsafe_project in the repository.)

STEP 1 — Create smartsafe_env.yml

Open Notepad (or any text editor) and save the file as:

%USERPROFILE%\smartsafe_project\smartsafe_env.yml

Paste the following YAML exactly:

name: smartsafe_env

channels:

- conda-forge
- defaults

dependencies:

- python=3.10
- pip
- numpy
- pandas
- scipy
- scikit-learn
- matplotlib
- jupyterlab
- notebook
- ipykernel
- joblib
- pillow
- rasterio
- geopandas
- shapely
- fiona
- pyproj
- contextily
- folium
- osmnx
- networkx
- requests
- seaborn
- tqdm
- wfdb

- sqlalchemy
- gdal
- pip:
 - opencv-python
 - imageio
 - descartes
 - torch
 - torchvision
 - rasterio[GDAL]

Note: conda-forge provides many GIS packages reliably. The rasterio[GDAL] pip entry is included to ensure pip-based rasterio variants if required by some configurations, but the conda installation route below will prefer conda packages first.

STEP 2 — Create the conda environment (Anaconda Prompt)

1. Open **Anaconda Prompt** (Search → *Anaconda Prompt* → Enter).
2. Change directory to the project folder:

```
cd "%USERPROFILE%\smartsafe_project"
```

3. Create the environment from the YAML:

```
conda env create -f smartsafe_env.yml
```

If conda env create fails, ensure smartsafe_env.yml is saved in the folder shown by the cd step and you have internet access. Errors often mean the YAML was not found or there are temporary package resolution issues.

4. Activate the environment:

```
conda activate smartsafe_env
```

5. Register a Jupyter kernel for this environment (so notebooks can pick it):

```
python -m ipykernel install --user --name smartsafe_env --display-name  
"smartsafe_env"
```

STEP 3 — Environment sanity check (Windows)

Purpose: verify key libraries import and report versions. With the environment active (conda activate smartsafe_env), run Python and paste the commands below *one block at a time*.

From Anaconda Prompt:

```
conda activate smartsafe_env
```

```
python
```

Inside the Python REPL, paste the following **exact** checks:

```
import sys
```

```
print("Python:", sys.version.split()[0])
```

```
import pandas as pd, numpy as np, sklearn, joblib
```

```
print("pandas:", pd.__version__)
```

```
print("numpy:", np.__version__)
```

```
print("scikit-learn:", sklearn.__version__)
```

```
print("joblib:", joblib.__version__)
```

```
import rasterio
```

```
print("rasterio:", rasterio.__version__)
```

```
import geopandas
```

```
print("geopandas:", geopandas.__version__)
```

```
import shapely, pyproj
```

```
print("shapely:", shapely.__version__)
```

```
print("pyproj:", pyproj.__version__)
```

```
import contextily, folium
```

```
print("contextily:", contextily.__version__)
```

```
print("folium:", folium.__version__)
```

```
import matplotlib
```

```
print("matplotlib:", matplotlib.__version__)
```

Then exit:

```
exit()
```

Expected outcome: each print statement shows a version string (no import errors). If all imports succeed, the environment is ready.

STEP 4 — Fix common GIS errors (GDAL / rasterio)

If you see errors during import rasterio or errors referencing GDAL, run **these** commands (while environment is active):

```
conda remove --force rasterio gdal -y
```

```
conda install -c conda-forge gdal rasterio -y
```

After running the commands, re-run the **Environment sanity check** above.

STEP 5 — Launch Jupyter Lab / Notebook

With the environment active:

```
conda activate smartsafe_env
```

```
jupyter lab
```

In the Jupyter interface, **open your notebook** and make sure to select the kernel:

Kernel -> Change kernel -> smartsafe_env

Now notebooks will execute using the smartsafe_env interpreter.

STEP 6 — Export the exact environment (recommended for sharing)

To create a reproducible environment file (no builds) for sharing:

```
conda env export --name smartsafe_env --no-builds > smartsafe_env_exported.yml
```

Place smartsafe_env_exported.yml in the project folder and commit it to the repository. This file captures exact package versions used on the machine where it was exported.

STEP 7 — Portable path & repo tips (important)

1. **Use relative paths within notebooks.** Example:

```
from pathlib import Path
```

```
PROJECT_ROOT = Path.cwd() # if notebook lives in project folder
```

```
DATA_DIR = PROJECT_ROOT / "terrain_data"
```

```
MODEL_PATH = PROJECT_ROOT / "cognitive_model.joblib"
```

2. **Do not hardcode usernames.** Replace absolute Windows user paths with %USERPROFILE% or relative repo paths.
3. **Add a .gitignore** to avoid committing large data and model binaries (suggestion lines):

```
__pycache__/
```

```
*.pyc
```

```
*.joblib
```

```
*.sqlite
```

```
*.db
```

```
.env
```

```
*.ipynb_checkpoints/
```

```
terrain_data/** # only if terrain_data is large - else keep sample data only
```

4. **Notebook execution:** if tests fail on another machine, ensure the Jupyter kernel is set to smartsafe_env and that the working directory is the project root.

Minimal troubleshooting checklist

- conda env create fails → check YAML file path and network; try conda env create -f .\smartsafe_env.yml from the project folder.
 - ImportError: GDAL → run the GDAL/rasterio reinstall commands in **STEP 4**.
 - sklearn import problems → ensure you are using the environment (conda activate smartsafe_env) and check python -m pip list vs conda list for duplicate installs.
 - Jupyter kernel not visible → re-run python -m ipykernel install --user --name smartsafe_env --display-name "smartsafe_env" inside the active environment.
-

Quick summary (one-page) Phase 0

1. Create %USERPROFILE%\smartsafe_project.
 2. Put required folders/files: cognitive_package_v2/, cognitive_model.joblib, terrain_data/, notebooks, smartsafe_env.yml.
 3. Create smartsafe_env.yml (use provided YAML).
 4. cd "%USERPROFILE%\smartsafe_project" → conda env create -f smartsafe_env.yml.
 5. conda activate smartsafe_env → python -m ipykernel install --user --name smartsafe_env --display-name "smartsafe_env".
 6. Run the sanity check Python commands (verify imports & versions).
 7. If GDAL/rasterio fail, run removal & reinstall from conda-forge.
 8. Launch jupyter lab, pick smartsafe_env, and run notebooks.
-

PHASES 1 → 5 — SmartSafe Project (Portable, Windows-friendly, Ready-to-run)

Assumption: Phase 0 (environment creation: smartsafe_env) is already completed and active.

Project root (portable): use a repo-relative folder or the environment variable %USERPROFILE% for Windows users. Throughout the document we use a repo-relative ./smartsafe_project path (recommended). Do **not** hardcode usernames.

project root

```
└─ cognitive_package_v2/      # your package (put model here if required)
└─ cognitive_model.joblib     # put here OR inside cognitive_package_v2 if package
expects it
└─ terrain_data/
└─ notebooks/
└─ phase1_download_dem.py
└─ phase2_compute_features.py
└─ phase3_raster_to_csv.py
└─ phase4_feature_engineering.py
```

└─ phase5_cognitive_integration.py

How to run (quick)

1. Open Anaconda Prompt (or terminal).
 2. cd to project root (the folder containing this repo).
 3. Activate the environment:
 4. conda activate smartsafe_env
 5. Run phases in order (each is independent code block / script):
 6. python phase1_download_dem.py
 7. python phase2_compute_features.py
 8. python phase3_raster_to_csv.py
 9. python phase4_feature_engineering.py
 10. python phase5_cognitive_integration.py
-

PHASE 1 — Places, SRTM download, HGT → GeoTIFF (DEM)

What this does:

- Defines the list of places (lat/lon).
- Downloads SRTM .hgt.gz tiles from the skadi AWS bucket.
- Decompresses .hgt and converts to GeoTIFF (EPSG:4326).
- Writes output to: terrain_data/<PlaceName>/<Tile>.tif

Where to run: project root.

File: phase1_download_dem.py (copy-paste exactly)

```
# phase1_download_dem.py
```

```
import os, requests, gzip, shutil
```

```
import numpy as np
```

```
import rasterio
```

```
from rasterio.transform import from_origin
```



```
# PLACE LIST (modify/add as needed)
```

```
places = [
```

```
    {"name": "Tawang_Arunachal_Pradesh", "lat": 27.586, "lon": 91.866},
    {"name": "Spiti_Valley_Himachal_Pradesh", "lat": 32.221, "lon": 78.071},
    {"name": "Valley_of_Flowers_Uttarakhand", "lat": 30.730, "lon": 79.610},
    {"name": "Kargil_Ladakh", "lat": 34.555, "lon": 76.133},
    {"name": "Gangtok_Sikkim", "lat": 27.338, "lon": 88.606}
```

```
]
```

```
def ensure_dir(path):
```

```
    os.makedirs(path, exist_ok=True)
```

```
def tile_name_from_latlon(lat, lon):
```

```
    ns = 'N' if lat >= 0 else 'S'
```

```
    ew = 'E' if lon >= 0 else 'W'
```

```
    return f"{ns}{abs(int(lat)):02d}{ew}{abs(int(lon)):03d}"
```

```
def download_hgt_for_tile(tile_name, out_dir, timeout=60):
```

```
    ensure_dir(out_dir)
```

```
    gz_name = f"{tile_name}.hgt.gz"
```

```
    gz_path = os.path.join(out_dir, gz_name)
```

```
    hgt_path = os.path.join(out_dir, f"{tile_name}.hgt")
```

```
    url = f"https://s3.amazonaws.com/elevation-tiles-  
prod/skadi/{tile_name[:3]}/{gz_name}"
```

```
    if os.path.exists(hgt_path):
```

```
        print(f"[INFO] Already have {hgt_path}")
```

```
        return hgt_path
```

```

print(f"[INFO] Downloading {url}")

r = requests.get(url, stream=True, timeout=timeout)

if r.status_code != 200:

    print(f"[WARN] HTTP {r.status_code} for {url}")

    return None


with open(gz_path, "wb") as f:

    for chunk in r.iter_content(1024*1024):

        if chunk:

            f.write(chunk)


# decompress

with gzip.open(gz_path, "rb") as f_in, open(hgt_path, "wb") as f_out:

    shutil.copyfileobj(f_in, f_out)

os.remove(gz_path)

print(f"[DONE] Decompressed {hgt_path}")

return hgt_path


def hgt_to_geotiff(hgt_path, tile_name, out_tif):

    with open(hgt_path, "rb") as f:

        data = f.read()


    # Detect size (3601 or 1201 commonly)

    for n in (3601, 1201):

        if len(data) == n*n*2:

            size = n

            break

```

else:

```
size = int(np.sqrt(len(data)//2))
```

```
print(f"[WARN] Unusual size, inferred {size}x{size}")
```

```
arr = np.frombuffer(data, dtype='>i2').reshape((size, size)).astype('float32')
```

```
arr[arr == -32768] = np.nan # SRTM voids to NaN
```

```
# parse tile_name e.g. N27E091
```

```
ns = tile_name[0]
```

```
lat_deg = int(tile_name[1:3])
```

```
ew = tile_name[3]
```

```
lon_deg = int(tile_name[4:7])
```

```
lat = lat_deg if ns == 'N' else -lat_deg
```

```
lon = lon_deg if ew == 'E' else -lon_deg
```

```
west = lon
```

```
north = lat + 1.0
```

```
res = 1.0 / (size - 1) # degrees per pixel
```

```
transform = from_origin(west, north, res, res)
```

```
meta = {
```

```
    "driver": "GTiff",
```

```
    "height": size,
```

```
    "width": size,
```

```
    "count": 1,
```

```
    "dtype": "float32",
```

```
    "crs": "EPSG:4326",
```

```
    "transform": transform,
```

```

        "nodata": np.nan
    }

    with rasterio.open(out_tif, "w", **meta) as dst:
        dst.write(arr, 1)

    print(f"[DONE] Wrote GeoTIFF: {out_tif}")

    return out_tif


# Run Phase 1
for p in places:

    folder = os.path.join("terrain_data", p["name"])
    ensure_dir(folder)

    tile = tile_name_from_latlon(p["lat"], p["lon"])
    hgt_path = download_hgt_for_tile(tile, folder)

    if hgt_path is None:

        print(f"[ERROR] tile not available for {p['name']}")
        continue

    tif_path = os.path.join(folder, f"{tile}.tif")

    if not os.path.exists(tif_path):

        hgt_to_geotiff(hgt_path, tile, tif_path)

    else:

        print(f"[INFO] GeoTIFF exists: {tif_path}")

```

Notes:

- If a location is on a tile edge you can download adjacent tiles and merge — I can provide an automatic merge script on request.
- Output path example: terrain_data/Tawang_Arunachal_Pradesh/N27E091.tif.

PHASE 2 — Compute slope & roughness rasters

What this does:

- Reads GeoTIFF DEMs and computes slope (in degrees) and a simple roughness map using a Laplacian-like high-pass filter.
- Writes slope.tif and rough.tif next to each DEM.

File: phase2_compute_features.py

```
# phase2_compute_features.py
```

```
import os
```

```
import numpy as np
```

```
import rasterio
```

```
from scipy import ndimage
```

```
def compute_slope(dem_tif, out_slope_tif):
```

```
    with rasterio.open(dem_tif) as src:
```

```
        dem = src.read(1, masked=True).filled(np.nan)
```

```
        transform = src.transform
```

```
        # pixel size in degrees (approx). Use metric reprojection for meter-accurate slope.
```

```
        res_x = transform.a
```

```
        res_y = -transform.e
```

```
        # compute gradients (handle NaNs)
```

```
        dem_f = np.where(np.isnan(dem), np.nan, dem)
```

```
        # replace NaNs with nearest for gradient calculation (preserve NaNs later)
```

```
        dem_filled = np.where(np.isnan(dem_f), np.nanmean(dem_f), dem_f)
```

```
        gy, gx = np.gradient(dem_filled, res_y, res_x)
```

```
        slope_rad = np.arctan(np.sqrt(gx**2 + gy**2))
```

```
        slope_deg = np.degrees(slope_rad)
```

```
        # restore NaNs where original DEM had NaN
```

```

slope_deg[np.isnan(dem_f)] = np.nan

profile = src.profile.copy()
profile.update(dtype='float32', nodata=np.nan)

with rasterio.open(out_slope_tif, "w", **profile) as dst:
    dst.write(slope_deg.astype('float32'), 1)
print(f"[DONE] {out_slope_tif}")
return out_slope_tif

def compute_roughness(dem_tif, out_rough_tif):
    with rasterio.open(dem_tif) as src:
        dem = src.read(1, masked=True).filled(np.nan)
        dem_f = np.nan_to_num(dem, nan=0.0)

        # roughness via 3x3 Laplacian / high-pass filter
        kernel = np.array([[1,1,1],[1,-8,1],[1,1,1]], dtype=float)
        rough = ndimage.convolve(dem_f, kernel, mode='nearest')
        rough = np.abs(rough)

        # set roughness to NaN where DEM was NaN
        rough[np.isnan(dem)] = np.nan

    profile = src.profile.copy()
    profile.update(dtype='float32', nodata=np.nan)
    with rasterio.open(out_rough_tif, "w", **profile) as dst:
        dst.write(rough.astype('float32'), 1)
    print(f"[DONE] {out_rough_tif}")

```

```
return out_rough_tif
```

```
# Run Phase 2 for all places
```

```
base = "terrain_data"
```

```
for place in os.listdir(base):
```

```
    folder = os.path.join(base, place)
```

```
    if not os.path.isdir(folder):
```

```
        continue
```

```
    tifs = [f for f in os.listdir(folder) if f.endswith(".tif")]
```

```
    dem_candidates = [f for f in tifs if "slope" not in f.lower() and "rough" not in f.lower()]
```

```
    if not dem_candidates:
```

```
        print(f"[WARN] No DEM found in {folder}")
```

```
        continue
```

```
    dem_path = os.path.join(folder, dem_candidates[0])
```

```
    slope_out = os.path.join(folder, "slope.tif")
```

```
    rough_out = os.path.join(folder, "rough.tif")
```

```
    if not os.path.exists(slope_out):
```

```
        compute_slope(dem_path, slope_out)
```

```
    else:
```

```
        print(f"[INFO] slope exists: {slope_out}")
```

```
    if not os.path.exists(rough_out):
```

```
        compute_roughness(dem_path, rough_out)
```

```
    else:
```

```
        print(f"[INFO] roughness exists: {rough_out}")
```

Notes:

- This script computes slope in degrees using degree-based pixel spacing. For meter-accurate slope, reproject DEM to a projected CRS (e.g. UTM) and compute gradient in meters — I can provide that if you want metric accuracy.
-

PHASE 3 — Convert rasters → point CSV (master dataset)

What this does:

- Converts rasters to a tabular CSV of valid pixels: (place, lat, lon, elevation, slope, roughness).
- Saves features.csv in each place folder and a merged terrain_master_dataset.csv at repo root.

File: phase3_raster_to_csv.py

```
# phase3_raster_to_csv.py
```

```
import os
```

```
import rasterio
```

```
import numpy as np
```

```
import pandas as pd
```

```
def raster_to_csv(dem_path, slope_path, rough_path, out_csv, place_name):
```

```
    with rasterio.open(dem_path) as dem_src, \
        rasterio.open(slope_path) as slope_src, \
        rasterio.open(rough_path) as rough_src:
```

```
        dem = dem_src.read(1, masked=True)
```

```
        slope = slope_src.read(1, masked=True)
```

```
        rough = rough_src.read(1, masked=True)
```

```
        rows, cols = np.where(~dem.mask)
```

```
        lons, lats = rasterio.transform.xy(dem_src.transform, rows, cols)
```

```
        data = {
```

```
            "place": place_name,
```

```
            "lat": lats,
```

```
            "lon": lons,
```

```
            "elevation": dem.data[rows, cols], # use .data to get raw values
```



```

        "slope": slope.data[rows, cols],
        "roughness": rough.data[rows, cols],
    }

    df = pd.DataFrame(data)

    df.to_csv(out_csv, index=False)

    print(f"[DONE] CSV: {out_csv}")

    return out_csv

```

```
base = "terrain_data"
```

```
master_list = []
```

```
for place in os.listdir(base):
```

```
    folder = os.path.join(base, place)
```

```
    if not os.path.isdir(folder):
```

```
        continue
```

```
    tifs = [f for f in os.listdir(folder) if f.endswith(".tif")]
```

```
    dem_candidates = [f for f in tifs if "slope" not in f.lower() and "rough" not in f.lower()]
```

```
    if not dem_candidates:
```

```
        print(f"[WARN] No DEM for", place)
```

```
        continue
```

```
    dem = os.path.join(folder, dem_candidates[0])
```

```
    slope = os.path.join(folder, "slope.tif")
```

```
    rough = os.path.join(folder, "rough.tif")
```

```
    out_csv = os.path.join(folder, "features.csv")
```

```
if os.path.exists(dem) and os.path.exists(slope) and os.path.exists(rough):
```

```
    raster_to_csv(dem, slope, rough, out_csv, place)
```

```
    master_list.append(pd.read_csv(out_csv))
```

```

else:

    print(f"[WARN] Missing rasters in {folder}")

if master_list:

    df_all = pd.concat(master_list, ignore_index=True)

    df_all.to_csv("terrain_master_dataset.csv", index=False)

    print("[DONE] Master dataset: terrain_master_dataset.csv (shape:
{}).format(df_all.shape))

else:

    print("[ERROR] No place CSVs created; check previous steps.")

```

Notes:

- Large DEMs produce large CSVs — sample or tile before converting if you want smaller datasets.
- Use relative paths in notebooks so code runs on any machine.

PHASE 4 — Feature engineering & aggregation per place

What this does:

- Cleans terrain_master_dataset.csv (drop NaNs, trim extreme outliers).
- Aggregates per place: mean/std/min/max of elevation, slope, roughness.
- Adds derived features: slope_risk (Low/Medium/High), normalized roughness.
- Saves terrain_engineered_dataset.csv.

File: phase4_feature_engineering.py

```

# phase4_feature_engineering.py

import pandas as pd

import numpy as np


df = pd.read_csv("terrain_master_dataset.csv")

print("Loaded master dataset:", df.shape)

```

```

# Drop rows missing key features
df = df.dropna(subset=["elevation", "slope", "roughness"])

# Remove extreme outliers (0.1% bottoms / tops)
for col in ["elevation", "slope", "roughness"]:
    lo = df[col].quantile(0.001)
    hi = df[col].quantile(0.999)
    df = df[(df[col] >= lo) & (df[col] <= hi)]

print("After cleaning:", df.shape)

agg_funcs = {
    "elevation": ["mean", "std", "min", "max"],
    "slope": ["mean", "std", "min", "max"],
    "roughness": ["mean", "std", "min", "max"],
}

df_engineered = df.groupby("place").agg(agg_funcs)
df_engineered.columns = ["_".join(c) for c in df_engineered.columns]
df_engineered = df_engineered.reset_index()

def slope_risk(mean_slope):
    if mean_slope < 15:
        return "Low"
    elif mean_slope < 30:
        return "Medium"
    else:
        return "High"

```

```

df_engineered["slope_risk"] = df_engineered["slope_mean"].apply(slope_risk)

# normalize roughness_mean to 0-1
rm = df_engineered["roughness_mean"]
df_engineered["roughness_norm"] = (rm - rm.min()) / (rm.max() - rm.min() + 1e-9)

df_engineered.to_csv("terrain_engineered_dataset.csv", index=False)
print("[DONE] terrain_engineered_dataset.csv")
print(df_engineered.head())

```

Notes:

- slope_risk thresholds are configurable. Adjust for your use-case.
- Keep the engineered CSV small (one row per place) for downstream models/visualizations.

PHASE 5 — Cognitive integration (attach cognitive scores per place)

What this does:

- Loads terrain_engineered_dataset.csv.
- Calls the provided cognitive_package_v2 (no changes to package) to generate cognitive predictions per place using a feature mapping.
- Produces terrain_with_cognitive.csv.

Important: Place cognitive_package_v2 and cognitive_model.joblib in the project root. If the package expects the model inside its folder, place cognitive_model.joblib in cognitive_package_v2/.

File: phase5_cognitive_integration.py

```

# phase5_cognitive_integration.py

import os

import pandas as pd

```

```

eng_path = "terrain_engineered_dataset.csv"

if not os.path.exists(eng_path):
    raise FileNotFoundError(f"{eng_path} not found — run Phase 4 first")

df_eng = pd.read_csv(eng_path)
print("Loaded engineered:", df_eng.shape)

# Try importing the cognitive package (do not change package internals)
try:
    import cognitive_package_v2.cognitive_estimator as cognitive_estimator
    print("[INFO] Imported cognitive_package_v2.cognitive_estimator")
except Exception as e:
    try:
        from cognitive_package_v2 import cognitive_estimator
        print("[INFO] Imported cognitive_package_v2 (module)")
    except Exception as e2:
        raise ImportError(
            "Could not import cognitive_package_v2.cognitive_estimator: "
            f"{e}\nMake sure 'cognitive_package_v2' folder is in project root and contains "
            "cognitive_estimator.py"
        )

def predict_with_package(feature_dict):
    """
    Attempts to call the package using common public functions.
    Returns a dict with keys: prediction, probability, and any raw output.
    """
    if hasattr(cognitive_estimator, "predict_from_dict"):

```

```

out = cognitive_estimator.predict_from_dict(feature_dict)

if isinstance(out, dict):
    return out

if isinstance(out, (list, tuple)) and len(out) and isinstance(out[0], dict):
    return out[0]

# fallback mapping
try:
    return {"prediction": int(out[0]), "probability": float(out[1]) if len(out)>1 else None}
except Exception:
    return {"prediction": out}

if hasattr(cognitive_estimator, "evaluate"):
    return cognitive_estimator.evaluate(**feature_dict)

if hasattr(cognitive_estimator, "predict_from_csv"):
    import tempfile, csv

    tmp = tempfile.NamedTemporaryFile(mode='w', delete=False, suffix=".csv")

    cols = list(feature_dict.keys())

    writer = csv.writer(tmp)

    writer.writerow(cols)

    writer.writerow([feature_dict[c] for c in cols])

    tmp.close()

    out = cognitive_estimator.predict_from_csv(tmp.name)

    os.unlink(tmp.name)

    return out

    raise AttributeError("No usable prediction function found in
cognitive_package_v2.cognitive_estimator")

mapped_results = []

```

```

for _, row in df_eng.iterrows():

    place = row["place"]

    # >>>> IMPORTANT: Replace the mapping keys below with the exact feature names
    # expected by your cognitive estimator.

    # This mapping is a placeholder that allows the script to run; update it to your
    # package's API.

    feature_dict = {

        "ECG_mean": float(row.get("elevation_mean", 0.0)),
        "ECG_std": float(row.get("elevation_std", 0.0)),
        "ECG_max": float(row.get("elevation_max", row.get("elevation_mean", 0.0))),
        "ECG_min": float(row.get("elevation_min", row.get("elevation_mean", 0.0))),
        "EMG_rms": float(row.get("roughness_mean", 0.0)),
        "GSR_hand_mean": float(row.get("roughness_mean", 0.0)),
        "GSR_foot_mean": float(row.get("roughness_mean", 0.0)),
        "HR_mean": float(row.get("slope_mean", 0.0)),
        "RESP_mean": float(row.get("slope_mean", 0.0))

    }

    try:

        pred = predict_with_package(feature_dict)

    except Exception as e:

        print(f"[WARN] Prediction failed for {place}: {e}")

        pred = {"prediction": None, "probability": None}

    mapped_results.append({

        "place": place,
        "cognitive_prediction": pred.get("prediction"),
        "cognitive_probability": pred.get("probability"),
        "cognitive_raw": pred

    })

```

```
df_cog = pd.DataFrame(mapped_results)

merged = df_eng.merge(df_cog, on="place", how="left")

merged.to_csv("terrain_with_cognitive.csv", index=False)

print("[DONE] Saved terrain_with_cognitive.csv")

print(merged.head())
```

Important integration notes:

- The feature_dict keys **must match exactly** what cognitive_package_v2 expects (the code above uses placeholder keys like ECG_mean, EMG_rms, etc.). If your package expects different names, update the feature_dict mapping before running Phase 5.
- If the package raises FileNotFoundError: cognitive_model.joblib, place cognitive_model.joblib either in project root or inside cognitive_package_v2/ depending on the package implementation.

Troubleshooting & tips (concise)

- **Conda create fails:** ensure smartsafe_env.yml is in the working folder and you have network access. Try conda env create -f .\smartsafe_env.yml.
- **GDAL / rasterio import errors:** reinstall from conda-forge:
 - conda remove --force rasterio gdal -y
 - conda install -c conda-forge gdal rasterio -y
- **Large CSVs or memory issues:** convert using a smaller window; process tiles, not whole country DEMs.
- **Jupyter kernel not visible:** re-run inside the active environment:
 - python -m ipykernel install --user --name smartsafe_env --display-name "smartsafe_env"
- **Cognitive package import error:** ensure cognitive_package_v2 is a package (contains __init__.py) or the project root is in PYTHONPATH. Running scripts from project root ensures package imports succeed.

Deliverables created by Phases 1–5

1. terrain_data/<PlaceName>/<Tile>.tif — DEM GeoTIFFs (Phase 1)
 2. terrain_data/<PlaceName>/slope.tif and rough.tif — feature rasters (Phase 2)
 3. terrain_data/<PlaceName>/features.csv — pixel-level CSV per place (Phase 3)
 4. terrain_master_dataset.csv — merged point-level CSV (Phase 3)
 5. terrain_engineered_dataset.csv — per-place engineered features (Phase 4)
 6. terrain_with_cognitive.csv — final dataset with cognitive predictions (Phase 5)
-

Final notes for the report / GitHub

- All paths in the scripts are **relative** to the project root — this ensures portability across machines.
 - **Do not** include personal usernames or laptop-specific paths. Use ./ or %USERPROFILE% if you must show absolute examples for Windows.
 - Keep terrain_data/ out of version control if it contains large binary data (use .gitignore entries for demonstration datasets). Commit only small sample tiles and the code.
 - If you want, I can produce a single **one-file Markdown** containing these phases formatted for GitHub, or a **PDF** suitable for printing. Tell me which and I'll produce it in this chat.
-

Repository layout

terrain_pipeline/

```
└─ pipeline_config.py
└─ step6_data_cleaning_and_feature_engineering.py
└─ step7_train_terrain_risk_model.py
└─ step8_build_risk_surfaces.py
└─ step9_raster_astar_cropped.py
└─ step10_multi_region_experiments.py
└─ run_pipeline.py
└─ tests/
```

```
|  └─ test_smoke_run.py
└─ requirements.txt
```

1) pipeline_config.py

(Shared configuration used by all steps. Keeps paths relative and easy to change.)

```
# pipeline_config.py
```

```
"""
```

Central config for terrain pipeline.

Edit only if you want to change base directories or filenames.

All paths are relative to repo root.

```
"""
```

```
from pathlib import Path
```

```
ROOT = Path(__file__).resolve().parent
```

```
# Input / Output directories
```

```
PLACES_DIR = ROOT / "terrain_data"    # expected folders:
terrain_data/<Place>/dem.tif, slope.tif, rough.tif
```

```
MASTER_CSV = ROOT / "terrain_master_dataset.csv"
```

```
ENGINEERED_CSV = ROOT / "terrain_engineered_dataset.csv"
```

```
MODEL_FILE = ROOT / "terrain_risk_model.joblib"
```

```
RESULTS_CSV = ROOT / "terrain_pathfinding_results.csv"
```

```
# Default runtime options
```

```
DEFAULT_PAD_METERS = 5000  # window padding for cropped A*
```

```
TRAIN_SAMPLE_FRAC = 0.25  # sampling fraction when dataset is large
```

```
RANDOM_STATE = 42
```

```
# Safety / performance toggles
```

```
MAX_TRAIN_ROWS = 200_000 # if master csv bigger, sample
```

```
USE_ML_MODEL_IF_PRESENT = True
```

2) step6_data_cleaning_and_feature_engineering.py

```
# step6_data_cleaning_and_feature_engineering.py
```

```
"""
```

Step 6: Data cleaning & feature engineering.

Produces: terrain_engineered_dataset.csv (one row per place).

Usage:

```
python step6_data_cleaning_and_feature_engineering.py
```

```
"""
```

```
import sys
```

```
import pandas as pd
```

```
import numpy as np
```

```
from pipeline_config import MASTER_CSV, ENGINEERED_CSV
```

```
from pathlib import Path
```

```
def clip_outliers(df, col, lower_pct=0.001, upper_pct=0.999):
```

```
    lo = df[col].quantile(lower_pct)
```

```
    hi = df[col].quantile(upper_pct)
```

```
    return df[(df[col] >= lo) & (df[col] <= hi)]
```

```
def slope_risk(mean_slope):
```

```
    if np.isnan(mean_slope):
```

```
        return "Unknown"
```

```

if mean_slope < 15:
    return "Low"

elif mean_slope < 30:
    return "Medium"

else:
    return "High"

def main():
    if not MASTER_CSV.exists():
        print(f"ERROR: master CSV not found at {MASTER_CSV}", file=sys.stderr)
        sys.exit(2)

    print("Loading master dataset:", MASTER_CSV)
    df = pd.read_csv(MASTER_CSV)
    print("Raw rows:", len(df))

    # Basic cleaning
    df = df.dropna(subset=["elevation", "slope", "roughness", "place"])
    print("After dropping NaNs:", len(df))

    # Outlier clipping column-wise
    for col in ["elevation", "slope", "roughness"]:
        if col in df.columns:
            df = clip_outliers(df, col)
    print("After outlier clipping:", len(df))

    # Aggregation
    agg_funcs = {

```

```

    "elevation": ["mean", "std", "min", "max"],
    "slope": ["mean", "std", "min", "max"],
    "roughness": ["mean", "std", "min", "max"]
}

df_eng = df.groupby("place").agg(agg_funcs)
df_eng.columns = ["_".join(c) for c in df_eng.columns]
df_eng = df_eng.reset_index()

# Derived features
df_eng["slope_risk"] = df_eng["slope_mean"].apply(slope_risk)
# normalized roughness (0..1)
rm = df_eng["roughness_mean"]
df_eng["roughness_norm"] = (rm - rm.min()) / (rm.max() - rm.min() + 1e-9)

ENGINEERED_CSV.parent.mkdir(parents=True, exist_ok=True)
df_eng.to_csv(ENGINEERED_CSV, index=False)
print("Saved engineered dataset:", ENGINEERED_CSV)
print(df_eng.head(10).to_string(index=False))

if __name__ == "__main__":
    main()

```

3) step7_train_terrain_risk_model.py

step7_train_terrain_risk_model.py

"""

Step 7: Train a simple terrain risk model (RandomForest).

Produces: terrain_risk_model.joblib

Usage:

```
python step7_train_terrain_risk_model.py
```

Notes:

- If you prefer rule-based risk, skip this step and step8 will fall back to rule-based behavior.

```
"""
```

```
import sys
```

```
import joblib
```

```
import numpy as np
```

```
import pandas as pd
```

```
from pipeline_config import MASTER_CSV, MODEL_FILE, TRAIN_SAMPLE_FRAC,  
RANDOM_STATE, MAX_TRAIN_ROWS
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.model_selection import train_test_split
```

```
def main():
```

```
    if not MASTER_CSV.exists():
```

```
        print(f"ERROR: master CSV not found at {MASTER_CSV}", file=sys.stderr)
```

```
        sys.exit(2)
```

```
    df = pd.read_csv(MASTER_CSV)
```

```
    df = df.dropna(subset=["elevation", "slope", "roughness"])
```

```
    if len(df) == 0:
```

```
        print("No usable rows after dropping NaNs.", file=sys.stderr)
```

```
        sys.exit(3)
```

```
    # Create simple heuristic label if no explicit label present
```

```
    df["roughness_norm"] = (df["roughness"] - df["roughness"].min()) /  
(df["roughness"].max() - df["roughness"].min() + 1e-9)
```

```

df["risk_label"] = ((df["slope"] >= 30) | (df["roughness_norm"] >= 0.8)).astype(int)

# Sample for speed if large
if len(df) > MAX_TRAIN_ROWS:
    sample = df.sample(frac=TRAIN_SAMPLE_FRAC, random_state=RANDOM_STATE)
else:
    sample = df

X = sample[["elevation", "slope", "roughness"]].values
y = sample["risk_label"].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=RANDOM_STATE, stratify=y)

print("Train size:", len(X_train), "Test size:", len(X_test))

clf = RandomForestClassifier(n_estimators=200, class_weight="balanced",
random_state=RANDOM_STATE, n_jobs=-1)

clf.fit(X_train, y_train)

print("Train acc:", clf.score(X_train, y_train), "Test acc:", clf.score(X_test, y_test))

MODEL_FILE.parent.mkdir(parents=True, exist_ok=True)

joblib.dump({"model": clf, "feature_columns": ["elevation", "slope", "roughness"]},
MODEL_FILE)

print("Saved terrain risk model to:", MODEL_FILE)

if __name__ == "__main__":
    main()

```

4) step8_build_risk_surfaces.py

```
# step8_build_risk_surfaces.py
```

```
"""
```

Step 8: Build per-pixel risk surfaces (GeoTIFF) for each place.

Reads dem.tif, slope.tif, rough.tif from each terrain_data/<Place>/ folder and writes risk.tif.

If a trained model exists at pipeline_config.MODEL_FILE it will be used; otherwise rule-based risk is used.

Usage:

```
python step8_build_risk_surfaces.py
```

```
"""
```

```
import os
```

```
import sys
```

```
import numpy as np
```

```
import joblib
```

```
import rasterio
```

```
from pipeline_config import PLACES_DIR, MODEL_FILE, USE_ML_MODEL_IF_PRESENT
```

```
from pathlib import Path
```

```
def rule_based_risk(slope_arr):
```

```
    """Map slope value -> penalty 0..1"""
```

```
    # slope in degrees
```

```
    out = np.full_like(slope_arr, np.nan, dtype='float32')
```

```
    mask = ~np.isnan(slope_arr)
```

```
    s = slope_arr[mask]
```

```
    out_vals = np.where(s < 15, 0.0, np.where(s < 30, 0.5, 1.0))
```

```
    out[mask] = out_vals
```

```
    return out
```



```

def compute_risk_for_tile(dem_tif, slope_tif, rough_tif, out_risk_tif,
model_bundle=None):

    with rasterio.open(dem_tif) as dsrc, rasterio.open(slope_tif) as ssrc,
rasterio.open(rough_tif) as rsrc:

        dem = dsrc.read(1, masked=True).filled(np.nan)
        slope = ssrc.read(1, masked=True).filled(np.nan)
        rough = rsrc.read(1, masked=True).filled(np.nan)

        risk = np.full_like(dem, np.nan, dtype='float32')
        valid_mask = ~np.isnan(dem)
        coords = np.where(valid_mask)

        if model_bundle:
            model = model_bundle["model"]
            vals = np.vstack([dem[coords], slope[coords], rough[coords]]).T
            preds = model.predict_proba(vals)[: , 1].astype('float32')
            risk[coords] = preds
        else:
            risk = rule_based_risk(slope).astype('float32')

        meta = dsrc.profile.copy()
        meta.update(dtype='float32', count=1, nodata=np.nan)
        out_dir = Path(out_risk_tif).parent
        out_dir.mkdir(parents=True, exist_ok=True)
        with rasterio.open(out_risk_tif, 'w', **meta) as dst:
            dst.write(risk, 1)

    return out_risk_tif

def main():

```

```

bundle = None

if USE_ML_MODEL_IF_PRESENT and MODEL_FILE.exists():
    print("Loading ML model:", MODEL_FILE)
    bundle = joblib.load(MODEL_FILE)
else:
    print("No ML model found. Using rule-based risk.")

if not PLACES_DIR.exists():
    print(f"ERROR: places directory not found: {PLACES_DIR}", file=sys.stderr)
    sys.exit(2)

for place_folder in sorted(os.listdir(PLACES_DIR)):
    folder = Path(PLACES_DIR) / place_folder
    if not folder.is_dir():
        continue
    dem = folder / "dem.tif"
    slope = folder / "slope.tif"
    rough = folder / "rough.tif"
    out_risk = folder / "risk.tif"
    if dem.exists() and slope.exists() and rough.exists():
        print("Processing:", folder)
        compute_risk_for_tile(str(dem), str(slope), str(rough), str(out_risk),
model_bundle=bundle)
        print("Saved:", out_risk)
    else:
        print("Skipping (missing rasters):", folder)

if __name__ == "__main__":

```

```
main()
```

5) step9_raster_astar_cropped.py

```
# step9_raster_astar_cropped.py
```

```
"""
```

Step 9: Risk-aware A* on a cropped raster window.

Provides functions:

- astar_on_array(cost, start_rc, goal_rc)
- crop_raster_window(src, start_lonlat, goal_lonlat, pad_meters)
- run_raster_astar_for_place(place_folder, start_lonlat, goal_lonlat, pad_meters)

Usage: import functions or run as script for quick test.

```
"""
```

```
import heapq
```

```
import math
```

```
import os
```

```
import numpy as np
```

```
import rasterio
```

```
from rasterio.windows import Window
```

```
from rasterio import transform as rio_transform
```

```
from pipeline_config import PLACES_DIR, DEFAULT_PAD_METERS
```

```
def heuristic(a, b):
```

```
    return math.hypot(a[0] - b[0], a[1] - b[1])
```

```
def astar_on_array(cost, start, goal, diagonal=True):
```

```
    rows, cols = cost.shape
```

```
    open_heap = []
```

```

heapq.heappush(open_heap, (0 + heuristic(start, goal), start))

came_from = {}

gscore = {start: 0}

neighs = [(-1,0),(1,0),(0,-1),(0,1)]

if diagonal:
    neighs += [(-1,-1),(-1,1),(1,-1),(1,1)]

while open_heap:
    _, current = heapq.heappop(open_heap)

    if current == goal:
        # reconstruct path
        p = []
        while current in came_from:
            p.append(current)
            current = came_from[current]
        p.append(start)
        return p[::-1]

    for dr, dc in neighs:
        nr, nc = current[0] + dr, current[1] + dc

        if 0 <= nr < rows and 0 <= nc < cols:
            if np.isnan(cost[nr, nc]) or np.isinf(cost[nr, nc]):
                continue

            dist = math.hypot(dr, dc)

            tentative = gscore[current] + dist * (1.0 + float(cost[nr, nc]))

            neighbor = (nr, nc)

            if tentative < gscore.get(neighbor, float('inf')):
                came_from[neighbor] = current

```

```
gscore[neighbor] = tentative
f = tentative + heuristic(neighbor, goal)
heapq.heappush(open_heap, (f, neighbor))

return None
```

```
def crop_raster_window(src, lon_lat_start, lon_lat_goal,
pad_meters=DEFAULT_PAD_METERS):

    # src: opened rasterio dataset

    # lon_lat_*: (lon, lat)

    s_col, s_row = src.index(lon_lat_start[0], lon_lat_start[1])
    g_col, g_row = src.index(lon_lat_goal[0], lon_lat_goal[1])

    xres = abs(src.transform.a)

    # approximate degrees->meters using mean latitude
    mean_lat = (lon_lat_start[1] + lon_lat_goal[1]) / 2.0
    meters_per_degree = 111_320 * math.cos(math.radians(mean_lat))
    pad_pixels = int(pad_meters / (xres * meters_per_degree) + 10)

    row_min = max(0, min(s_row, g_row) - pad_pixels)
    row_max = min(src.height, max(s_row, g_row) + pad_pixels)
    col_min = max(0, min(s_col, g_col) - pad_pixels)
    col_max = min(src.width, max(s_col, g_col) + pad_pixels)

    height = row_max - row_min
    width = col_max - col_min

    window = Window(col_off=col_min, row_off=row_min, width=width, height=height)
    arr = src.read(1, window=window, masked=True).filled(np.nan)
```

```

start_in_window = (s_row - row_min, s_col - col_min)
goal_in_window = (g_row - row_min, g_col - col_min)
transform = src.window_transform(window)
return arr, transform, start_in_window, goal_in_window

def run_raster_astar_for_place(place_folder, start_lonlat, goal_lonlat,
pad_meters=DEFAULT_PAD_METERS):

    folder = os.path.join(PLACES_DIR, place_folder)

    risk_path = os.path.join(folder, "risk.tif")

    if not os.path.exists(risk_path):

        raise FileNotFoundError(f"Missing risk raster: {risk_path}")

    with rasterio.open(risk_path) as src:

        arr, transform, start_rc, goal_rc = crop_raster_window(src, start_lonlat, goal_lonlat,
pad_meters=pad_meters)

        if np.isnan(arr).all():

            print("Cropped array is all NaN (no-data). Increase pad or check input rasters.")

            return None

        path_rc = astar_on_array(arr, start_rc, goal_rc)

        if not path_rc:

            print("No path found")

            return None

        # convert rc -> (lat, lon)

        path_latlon = [ rasterio.transform.xy(transform, r, c, offset='center')[::-1] for r,c in
path_rc ]

        return path_latlon

# Optional: quick CLI test

if __name__ == "__main__":

```

```

import argparse

parser = argparse.ArgumentParser(description="Small test for raster A* (place folder
and lon/lat start/goal)")

parser.add_argument("place", help="place folder name under terrain_data")
parser.add_argument("slon", type=float); parser.add_argument("slat", type=float)
parser.add_argument("glon", type=float); parser.add_argument("glat", type=float)

args = parser.parse_args()

res = run_raster_astar_for_place(args.place, (args.slon, args.slat), (args.glon,
args.glat))

if res:

    print("Path length points:", len(res))

else:

    print("No path returned.")

```

6) step10_multi_region_experiments.py

```
# step10_multi_region_experiments.py
```

```
"""
```

Step 10: Multi-region experiments - compare shortest vs risk-aware A*.

Produces a CSV with metrics and saves per-segment folium HTML maps.

Usage:

```
python step10_multi_region_experiments.py
```

```
"""
```

```
import os
```

```
import numpy as np
```

```
import pandas as pd
```

```
import folium
```

```
import rasterio
```

```
from math import hypot
```

```
from pipeline_config import RESULTS_CSV, PLACES_DIR, DEFAULT_PAD_METERS
from step9_raster_astar_cropped import run_raster_astar_for_place, astar_on_array
from rasterio import transform as rio_transform
```

```
# Define your segments here (place, start_lon, start_lat, goal_lon, goal_lat)
```

```
SEGMENTS = [
```

```
    # examples (replace with your own)
```

```
    ("Tawang_Arunachal_Pradesh", 91.866, 27.586, 91.925, 27.630),
```

```
    ("Spiti_Valley_Himachal_Pradesh", 78.071, 32.221, 78.15, 32.28),
```

```
    ("Valley_of_Flowers_Uttarakhand", 79.61, 30.73, 79.67, 30.76),
```

```
    ("Kargil_Ladakh", 76.133, 34.555, 76.20, 34.60),
```

```
    ("Gangtok_Sikkim", 88.606, 27.338, 88.66, 27.36),
```

```
]
```

```
def rc_list_to_lonlat(rc_list, transform):
```

```
    return [ rasterio.transform.xy(transform, r, c, offset='center')[::-1] for r,c in rc_list ]
```

```
def shortest_path_on_window(arr, start_rc, goal_rc):
```

```
    # arr: window cost (but for shortest we use zeros)
```

```
    zero_cost = np.zeros_like(arr, dtype=float)
```

```
    return astar_on_array(zero_cost, start_rc, goal_rc)
```

```
def run_segment(segment, pad_meters=DEFAULT_PAD_METERS):
```

```
    place, slon, slat, glon, glat = segment
```

```
    folder = os.path.join(PLACES_DIR, place)
```

```
    risk_path = os.path.join(folder, "risk.tif")
```

```
    if not os.path.exists(risk_path):
```

```
        print("Missing risk for", place); return None
```



```

with rasterio.open(risk_path) as src:

    arr, transform, start_rc, goal_rc = src.read(1, window=None,
masked=True).filled(np.nan), src.transform, None, None

    # Use crop helper from step9 to avoid repeating code

    # Call run_raster_astar_for_place (which will crop)

    risk_path_latlon = run_raster_astar_for_place(place, (slon, slat), (glon, glat),
pad_meters=pad_meters)

    # For shortest path we replicate crop logic to get start/goal in window and run A* with
zero cost

    # Simpler approach: if risk_path_latlon is None, skip
    if risk_path_latlon is None:

        print("Risk-aware planner failed for", place); return None


    # To compute shortest baseline, re-open and crop to same pad
    from step9_raster_astar_cropped import crop_raster_window, astar_on_array

    with rasterio.open(risk_path) as src:

        arr_crop, transform, start_rc, goal_rc = crop_raster_window(src, (slon, slat), (glon,
glat), pad_meters=pad_meters)

        if arr_crop is None:

            return None

        short_rc = astar_on_array(np.zeros_like(arr_crop, dtype=float), start_rc, goal_rc)

        if short_rc is None:

            short_latlon = []

            short_length = None

        else:

            short_latlon = [ rasterio.transform.xy(transform, r, c, offset='center')[::-1] for r,c in
short_rc ]

            short_length = sum(hypot(short_latlon[i+1][0]-short_latlon[i][0],
short_latlon[i+1][1]-short_latlon[i][1])

```

```

        for i in range(len(short_latlon)-1))

    risk_latlon = risk_path_latlon

    risk_length = sum(hypot(risk_latlon[i+1][0]-risk_latlon[i][0], risk_latlon[i+1][1]-
risk_latlon[i][1])

        for i in range(len(risk_latlon)-1)) if len(risk_latlon) > 1 else None

    # average risk on risk path (re-sample arr_crop at path indices)

    avg_risk = None

    try:

        path_rc_indices = [(int(round((p[0] - transform[5]) / transform[4])), int(round((p[1] -
transform[2]) / transform[0]))) for p in risk_latlon]

    except Exception:

        avg_risk = None

    # Save folium map

    m = folium.Map(location=[slat, slon], zoom_start=12)

    folium.PolyLine(short_latlon, color="blue", weight=3, opacity=0.8,
tooltip="Shortest").add_to(m)

    folium.PolyLine(risk_latlon, color="red", weight=3, opacity=0.8, tooltip="Risk-
Aware").add_to(m)

    folium.Marker([slat, slon], popup="Start").add_to(m)

    folium.Marker([glat, glon], popup="Goal").add_to(m)

    out_html = f"{place}_comparison.html"

    m.save(out_html)

    return {

        "place": place,

        "risk_path_points": len(risk_latlon),

```

```

    "risk_length_approx_deg": risk_length,
    "avg_risk": avg_risk,
    "short_path_points": len(short_latlon),
    "short_length_approx_deg": short_length,
    "map_file": out_html
}

def main():
    rows = []
    for seg in SEGMENTS:
        print("Running segment:", seg[0])
        res = run_segment(seg)
        if res:
            rows.append(res)
    if rows:
        df = pd.DataFrame(rows)
        df.to_csv(RESULTS_CSV, index=False)
        print("Saved results to", RESULTS_CSV)
        print(df)
    else:
        print("No results to save.")

if __name__ == "__main__":
    main()

```

Note: step10 includes example segments — replace or extend SEGMENTS with the start/goal pairs you wish to test.

7) run_pipeline.py (wrapper)

```
# run_pipeline.py
```

```
"""
```

Light wrapper to run Steps 6-10 in sequence.

Usage:

```
python run_pipeline.py --steps 6 7 8 9 10
```

Or:

```
python run_pipeline.py --all
```

This wrapper uses the per-step scripts as modules (imports).

```
"""
```

```
import argparse
```

```
import subprocess
```

```
import sys
```

```
from pathlib import Path
```

```
ROOT = Path(__file__).resolve().parent
```

```
STEP_FILES = {
```

```
    6: "step6_data_cleaning_and_feature_engineering.py",
```

```
    7: "step7_train_terrain_risk_model.py",
```

```
    8: "step8_build_risk_surfaces.py",
```

```
    9: "step9_raster_astar_cropped.py",
```

```
    10: "step10_multi_region_experiments.py"
```

```
}
```

```
def run_step(step_num):
```

```
    fname = ROOT / STEP_FILES[step_num]
```

```
    print(f"Running Step {step_num}: {fname.name}")
```

```

# Run as a subprocess so it behaves like running the .py directly.
res = subprocess.run([sys.executable, str(fname)], capture_output=False)
if res.returncode != 0:
    print(f"Step {step_num} failed with return code {res.returncode}")
    return False
return True

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--steps", nargs="+", type=int, help="space separated step
numbers to run (6..10)")
    parser.add_argument("--all", action="store_true", help="run all steps 6..10")
    args = parser.parse_args()

    to_run = []
    if args.all:
        to_run = sorted(STEP_FILES.keys())
    elif args.steps:
        to_run = args.steps
    else:
        parser.print_help()
        return

    for s in to_run:
        if s not in STEP_FILES:
            print("Invalid step:", s)
            continue
        ok = run_step(s)

```

```

    if not ok:

        print("Aborting pipeline due to failure in step", s)

        break

if __name__ == "__main__":

    main()

```

8) tests/test_smoke_run.py

(very small smoke test — checks that scripts are importable and config paths exist)

```
# tests/test_smoke_run.py
```

```
"""
```

Minimal smoke tests. Run with pytest (optional).

These are not full functional tests (they don't run raster processing),
they verify module import and config accessibility.

```
"""
```

```
import importlib
```

```
import pipeline_config
```

```
import os
```

```
def test_config_import():
```

```
    assert pipeline_config.PLACES_DIR is not None
```

```
def test_step_imports():
```

```
    modules = [
```

```
        "step6_data_cleaning_and_feature_engineering",
```

```
        "step7_train_terrain_risk_model",
```

```
        "step8_build_risk_surfaces",
```

```
"step9_raster_astar_cropped",  
"step10_multi_region_experiments",  
]  
  
for m in modules:  
    importlib.import_module(m)
```

9) requirements.txt

(install in environment)

pandas

numpy

rasterio

scikit-learn

joblib

folium

pytest

(You can pin versions if required for reproducibility.)

How to run (commands to put in your report)

Place all files in a folder terrain_pipeline/ and ensure your project has the terrain_data/<Place>/dem.tif, slope.tif, rough.tif files and terrain_master_dataset.csv (from steps 1–5). Then:

1. Create environment & install requirements (example with conda/pip):

```
conda create -n smartsafe_env python=3.10 -y
```

```
conda activate smartsafe_env
```

```
pip install -r requirements.txt
```

2. Run Step 6:

```
python step6_data_cleaning_and_feature_engineering.py
```

3. (Optional) Train model (Step 7):

```
python step7_train_terrain_risk_model.py
```

4. Build risk surfaces (Step 8):

```
python step8_build_risk_surfaces.py
```

5. Test one cropped A* run (Step 9 CLI example):

```
python step9_raster_astar_cropped.py "Tawang_Arunachal_Pradesh" 91.866 27.586  
91.925 27.63
```

6. Run multi-region experiments (Step 10):

```
python step10_multi_region_experiments.py
```

7. Or run all steps with the wrapper:

```
python run_pipeline.py --all
```

8. (Optional) Run smoke tests:

```
pytest -q
```

Small implementation notes / tips for your report

- All paths are relative; the code creates output directories where needed — dropping the folder onto any laptop (with Python + the packages) should run identically.
- If machine memory is low, reduce `DEFAULT_PAD_METERS` in `pipeline_config.py` or pass a lower pad into the A* calls.
- For accurate distance in meters convert lat/lon degree differences using `pyproj.Geod` if you need metric lengths (we left degrees for simplicity; comment is in the scripts).
- If you prefer purely rule-based risk, skip step 7 — the Step 8 script will auto-fall back to rule-based logic when no model exists.
- No personal identifiers or usernames are present — filenames and logs are generic.

Phase 11 → Phase 16: Risk-aware Terrain Pathfinding — Report & Code

Overview

This document describes the remaining pipeline (Phases 11 → 16) for producing risk-aware safe routes that fuse cognitive predictions with terrain features and run windowed A* pathfinding. The pipeline is designed to avoid memory issues by planning

on cropped windows, using a coarse→refine approach, and chaining multi-hop segments. All code blocks are runnable in `smartsafe_env` (Python 3.x) and assume required inputs are prepared by earlier phases:

Required input files (expected)

- `terrain_data/<PlaceFolder>/slope.tif` — slope raster (EPSG:4326)
- `terrain_data/<PlaceFolder>/rough.tif` — roughness raster (aligned to `slope.tif`)
- `terrain_with_cognitive_fixed.csv` — CSV columns: `place`, `lat`, `lon`, `cognitive_score` (or similar)
- `smartsafe_env` Python environment with `rasterio`, `numpy`, `pandas`, `pyproj`, `joblib`, `folium`, `matplotlib` installed
- `cognitive_package_v2/` (kept as-is) and optionally `cognitive_model.joblib` if needed

Design decisions

- Work per-segment (cropped windows) to limit memory usage.
- Coarse planning (resample) for speed, refine locally for accuracy.
- Cost model = (meters traveled) × (penalty), where $\text{penalty} = 1 + \alpha \times \text{terrain_risk}$ and `terrain_risk` is derived from slope & roughness.
- Heuristic = geodesic distance (meters) to keep metric units consistent.

0 — Preliminaries (run once)

Copy and run the following in your notebook or script top cell.

```
# Phase 11 start: Preliminaries
```

```
import os
```

```
import math
```

```
import joblib
```

```
import heapq
```

```
import json
```

```
import time
```

```
import rasterio
```

```
import numpy as np
```

```

import pandas as pd

from rasterio.windows import Window

from rasterio.transform import Affine

from rasterio.transform import rowcol

from pyproj import Geod

# Geodesic helper (meters) - accurate
_geod = Geod(ellps="WGS84")

def haversine_m(lat1, lon1, lat2, lon2):
    # returns geodesic distance in meters between two lat/lon points
    az12, az21, dist = _geod.inv(lon1, lat1, lon2, lat2)
    return dist

# Safety: ensure warnings show inline
import warnings

warnings.filterwarnings("default")

```

1 — Step 11: Build fused cellwise risk window

Purpose: crop a window around a center location, compute a terrain-derived risk multiplier from slope and rough rasters, return a penalty array and raster transform for A*.

Key parameters

- `pad_km`: half-window size in kilometers (e.g., 5 → total 10 km window)
- `resample_factor`: integer downsampling factor for coarse planning (≥ 1)

Step 11: build_fused_risk_window

```

def build_fused_risk_window(slope_tif, rough_tif, center_lat, center_lon, pad_km=5,
resample_factor=1):

```

```

    """

```

Returns: (terrain_penalty, transform)

- terrain_penalty: 2D numpy array (rows,cols), dtype=float, values >=1.0 (np.nan for masked/invalid)

- transform: affine transform for the returned array (rasterio Affine)

"""

pad_m = float(pad_km) * 1000.0

Open slope to compute pixel indexing and window bounds

with rasterio.open(slope_tif) as src:

convert center lat/lon -> row,col in source raster

try:

center_col, center_row = ~src.transform * (center_lon, center_lat)

round to center pixel

center_row = int(round(center_row))

center_col = int(round(center_col))

except Exception:

fallback to src.index

center_row, center_col = src.index(center_lon, center_lat)

estimate meters per pixel using small offset horizontally (safe for small windows)

method: sample two adjacent pixel centers to estimate distance

h = 1

ensure indices inside image

r0 = np.clip(center_row, 0, src.height - 1)

c0 = np.clip(center_col, 0, src.width - 1)

r1 = r0

c1 = np.clip(c0 + h, 0, src.width - 1)

lon0, lat0 = src.transform * (c0 + 0.5, r0 + 0.5)

lon1, lat1 = src.transform * (c1 + 0.5, r1 + 0.5)

```

meters_per_pixel = max(1e-6, haversine_m(lat0, lon0, lat1, lon1))

pad_pixels = int(math.ceil(pad_m / meters_per_pixel))

# compute window bounds
row_min = max(0, center_row - pad_pixels)
row_max = min(src.height - 1, center_row + pad_pixels)
col_min = max(0, center_col - pad_pixels)
col_max = min(src.width - 1, center_col + pad_pixels)

width = col_max - col_min + 1
height = row_max - row_min + 1

window = Window(col_off=col_min, row_off=row_min, width=width, height=height)
slope_arr = src.read(1, window=window, masked=True).filled(np.nan)
src_transform = src.window_transform(window)

# read roughness from same window (assumes alignment)
with rasterio.open(rough_tif) as src2:
    rough_arr = src2.read(1, window=Window(col_min, row_min, width, height),
masked=True).filled(np.nan)

# downsample if requested (coarse planning)
if resample_factor > 1:
    slope_arr = slope_arr[:, ::resample_factor, ::resample_factor]
    rough_arr = rough_arr[:, ::resample_factor, ::resample_factor]

    # scale transform: equivalent to increasing pixel size by factor
    new_transform = src_transform * Affine.scale(resample_factor)

```

```

else:
    new_transform = src_transform

# compute normalized terrain risk (tunable)
# slope: clamp 0..60 degrees -> normalized 0..1
slope_norm = np.clip(slope_arr, 0, 60) / 60.0
rough_norm = np.nan_to_num(rough_arr, nan=0.0)
# normalize roughness by 95th percentile (robust)
if np.all(np.isnan(rough_arr)):
    r95 = 1.0
else:
    r95 = float(np.nanpercentile(rough_norm, 95))
    if r95 <= 0.0:
        r95 = 1.0
rough_norm = np.clip(rough_norm / r95, 0.0, 1.0)

terrain_risk = 0.6 * slope_norm + 0.4 * rough_norm

# convert into multiplicative penalty: penalty = 1 + alpha * terrain_risk
alpha = 2.0 # tuning knob: larger -> stronger avoidance of steep/rough
terrain_penalty = 1.0 + alpha * terrain_risk

# mask out nan cells to np.nan (A* should skip these)
terrain_penalty = terrain_penalty.astype(float)
terrain_penalty[np.isnan(slope_arr) & np.isnan(rough_arr)] = np.nan

return terrain_penalty, new_transform

```

Usage example

```

place_folder = "Tawang_Arunachal_Pradesh"
slope_tif = f"terrain_data/{place_folder}/slope.tif"
rough_tif = f"terrain_data/{place_folder}/rough.tif"
center_lat, center_lon = 27.600, 91.88

penalty_window, transform = build_fused_risk_window(slope_tif, rough_tif, center_lat,
center_lon, pad_km=8, resample_factor=2)

print("risk window shape:", penalty_window.shape)

```

2 — Step 12: Windowed risk-aware A* (grid A*)

Purpose: A* planner that operates on the terrain_penalty grid and uses geodesic distances for both movement costs and heuristic.

Notes:

- The cost to move from cell A to B is $\text{distance_meters}(A,B) * \text{penalty_at_B}$.
- Heuristic uses geodesic (meters) to goal to preserve admissibility (if penalty ≥ 1 , heuristic remains admissible for pure distance; using it with penalty means heuristic is optimistic but still effective).
- Supports 8-connected neighbors.

Step 12: risk-aware A* on a window

```

def astar_risk(terrain_penalty, transform, start_latlon, goal_latlon,
max_iters_factor=10):

```

```

    """

```

```

    terrain_penalty: 2D numpy array, penalty  $\geq 1.0$ , np.nan for blocked cells

```

```

    transform: affine transform for the array

```

```

    start_latlon: (lat, lon)

```

```

    goal_latlon: (lat, lon)

```

```

    returns: list of (row, col) indices representing path, or None

```

```

    """

```

```

    rows, cols = terrain_penalty.shape

```

```
def latlon_to_rc(lat, lon):  
    # ~transform maps lon,lat -> col,row in array coordinates  
    colf, rowf = ~transform * (lon, lat)  
    return int(round(rowf)), int(round(colf))
```

```
def rc_to_latlon(rc):  
    r, c = rc  
    # transform*(col + 0.5, row + 0.5) -> (lon, lat)  
    lon, lat = transform * (c + 0.5, r + 0.5)  
    return lat, lon
```

```
start_rc = latlon_to_rc(*start_latlon)  
goal_rc = latlon_to_rc(*goal_latlon)
```

```
def in_bounds(rc):  
    r, c = rc  
    return 0 <= r < rows and 0 <= c < cols
```

```
def heuristic(a, b):  
    lat1, lon1 = rc_to_latlon(a)  
    lat2, lon2 = rc_to_latlon(b)  
    return haversine_m(lat1, lon1, lat2, lon2)
```

```
# Initialize A* structures
```

```
open_heap = []
```

```
gscore = {}
```

```
fscore = {}
```

```
came_from = {}
```

```
start = start_rc
```

```
goal = goal_rc
```

```
if not in_bounds(start) or not in_bounds(goal):
```

```
    return None
```

```
# Start might be on a blocked pixel; attempt to nudge to nearest valid neighbor
```

```
def valid_or_none(rc):
```

```
    r,c = rc
```

```
    if not in_bounds(rc):
```

```
        return False
```

```
    val = terrain_penalty[r,c]
```

```
    return not (np.isnan(val) or np.isinf(val))
```

```
if not valid_or_none(start):
```

```
    # try small neighborhood to find nearest valid cell
```

```
    found = None
```

```
    for radius in [1,2,3,5]:
```

```
        for dr in range(-radius, radius+1):
```

```
            for dc in range(-radius, radius+1):
```

```
                rr = start[0] + dr; cc = start[1] + dc
```

```
                if in_bounds((rr,cc)) and valid_or_none((rr,cc)):
```

```
                    found = (rr,cc); break
```

```
            if found: break
```

```
        if found:
```

```
            start = found; break
```

```
    if not found:
```



```
    return None
```

```
if not valid_or_none(goal):
```

```
    # same attempt for goal
```

```
    found = None
```

```
    for radius in [1,2,3,5]:
```

```
        for dr in range(-radius, radius+1):
```

```
            for dc in range(-radius, radius+1):
```

```
                rr = goal[0] + dr; cc = goal[1] + dc
```

```
                if in_bounds((rr,cc)) and valid_or_none((rr,cc)):
```

```
                    found = (rr,cc); break
```

```
            if found: break
```

```
    if found:
```

```
        goal = found; break
```

```
    if not found:
```

```
        return None
```

```
gscore[start] = 0.0
```

```
fscore[start] = heuristic(start, goal)
```

```
heapq.heappush(open_heap, (fscore[start], start))
```

```
neighbors = [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(-1,1),(1,-1),(1,1)]
```

```
max_iters = rows * cols * max_iters_factor
```

```
iters = 0
```

```
while open_heap:
```

```
    iters += 1
```

```

if iters > max_iters:
    break

_, current = heapq.heappop(open_heap)

if current == goal:
    # reconstruct
    path = []
    cur = current
    while cur in came_from:
        path.append(cur)
        cur = came_from[cur]
    path.append(start)
    return path[::-1]

cr, cc = current
for dr, dc in neighbors:
    nr, nc = cr + dr, cc + dc
    neighbor = (nr, nc)
    if not in_bounds(neighbor):
        continue
    # skip blocked
    cost_mult = terrain_penalty[nr, nc]
    if np.isnan(cost_mult) or np.isinf(cost_mult):
        continue
    latc, lonc = rc_to_latlon(current)
    latn, lonn = rc_to_latlon(neighbor)
    step_dist = haversine_m(latc, lonc, latn, lonn)
    tentative_g = gscore[current] + step_dist * float(cost_mult)
    if tentative_g < gscore.get(neighbor, float("inf")):

```

```

    came_from[neighbor] = current
    gscore[neighbor] = tentative_g
    f = tentative_g + heuristic(neighbor, goal)
    heapq.heappush(open_heap, (f, neighbor))

return None

```

Example usage

```

start_latlon = (27.586, 91.866) # (lat, lon)
goal_latlon = (27.630, 91.925)
center_lat = 0.5*(start_latlon[0] + goal_latlon[0])
center_lon = 0.5*(start_latlon[1] + goal_latlon[1])

penalty, transform = build_fused_risk_window(slope_tif, rough_tif, center_lat,
center_lon, pad_km=6, resample_factor=2)

path_rc = astar_risk(penalty, transform, start_latlon, goal_latlon)
print("cells in path:", len(path_rc) if path_rc else "No path")

```

Tuning tips

- If None returned, increase pad_km to include both start and goal or verify the appropriate tile is used.
- For large inter-point distances: use resample_factor > 1 (coarse planning) then refine on bounding box around coarse path.

3 — Step 13: Batch experiments (multi-pair runs)

Purpose: run many start-goal pairs, compare risk-aware vs shortest (distance-only), refine results, and save metrics.

Behavior

- For each pair: coarse run with resample_coarse → optional refine at resample=1 around coarse path bounding box.
- Baseline uses constant penalty (=1) to compute shortest-path by distance.

Step 13: run_experiments_batch

```
def run_experiments_batch(place_rows_df, pairs, pad_km=6, resample_coarse=4,  
partial_save="batch_path_results_partial.csv"):
```

```
    """
```

```
    place_rows_df: DataFrame with at least columns ['place','lat','lon']
```

```
    pairs: list of (placeA, placeB) tuples referencing place column values
```

```
    returns: pandas DataFrame with results for each pair
```

```
    """
```

```
    results = []
```

```
    for a,b in pairs:
```

```
        try:
```

```
            rowA = place_rows_df[place_rows_df["place"]==a].iloc[0]
```

```
            rowB = place_rows_df[place_rows_df["place"]==b].iloc[0]
```

```
        except Exception:
```

```
            print(f"Skipping pair {a}->{b}: missing place info")
```

```
            continue
```

```
        start = (float(rowA["lat"]), float(rowA["lon"]))
```

```
        goal = (float(rowB["lat"]), float(rowB["lon"]))
```

```
        # choose raster folder heuristically (start's folder). If your tiles span multiple folders,
```

```
        # consider selecting the tile containing both start and goal or pre-merging tiles.
```

```
        folder = a.replace(",","").replace(" ","_")
```

```
        slope_tif = f"terrain_data/{folder}/slope.tif"
```

```
        rough_tif = f"terrain_data/{folder}/rough.tif"
```

```
        if not (os.path.exists(slope_tif) and os.path.exists(rough_tif)):
```

```
            print(f"Missing rasters for {a} - {slope_tif}")
```

```
            results.append({"start":a,"end":b,"used":"missing_raster"})
```

```

        continue

# coarse fused penalty
center_lat = 0.5*(start[0] + goal[0])
center_lon = 0.5*(start[1] + goal[1])
try:
    penalty_coarse, t_coarse = build_fused_risk_window(slope_tif, rough_tif,
center_lat, center_lon, pad_km=pad_km, resample_factor=resample_coarse)
except Exception as e:
    print(f"Error building coarse window for {a}->{b}: {e}")
    results.append({"start":a,"end":b,"used":"window_error"})
    continue

path_coarse = astar_risk(penalty_coarse, t_coarse, start, goal)

# baseline (distance-only) on same coarse grid
if path_coarse:
    ones = np.ones_like(penalty_coarse)
    baseline_path = astar_risk(ones, t_coarse, start, goal)
else:
    baseline_path = None

# refine near coarse path (if found)
refined_path = None
if path_coarse:
    # compute bounding box in latlon of coarse path and expand small buffer
    lats, lons = [], []
    for r,c in path_coarse:

```

```

lonp, latp = t_coarse * (c + 0.5, r + 0.5)

lats.append(latp); lons.append(lonp)

bb_lat_min = max(min(lats) - 0.02, -90); bb_lat_max = min(max(lats) + 0.02, 90)
bb_lon_min = max(min(lons) - 0.02, -180); bb_lon_max = min(max(lons) + 0.02,
180)

center_lat = 0.5*(bb_lat_min + bb_lat_max)

center_lon = 0.5*(bb_lon_min + bb_lon_max)

try:

    penalty_fine, t_fine = build_fused_risk_window(slope_tif, rough_tif, center_lat,
center_lon, pad_km=2, resample_factor=1)

    refined_path = astar_risk(penalty_fine, t_fine, start, goal)

except Exception:

    refined_path = None


# helper metrics

def path_metrics(path_rc, transform, penalty):

    if not path_rc:

        return None, None

    dsum = 0.0

    risks = []

    for i in range(len(path_rc)-1):

        r1,c1 = path_rc[i]; r2,c2 = path_rc[i+1]

        lon1, lat1 = transform * (c1 + 0.5, r1 + 0.5)

        lon2, lat2 = transform * (c2 + 0.5, r2 + 0.5)

        d = haversine_m(lat1, lon1, lat2, lon2)

        dsum += d

    for (r,c) in path_rc:

        val = penalty[r,c] if (0 <= r < penalty.shape[0] and 0 <= c < penalty.shape[1]) else

```

```

        if np.isnan(val):
            val = 1.0

        risks.append(val)

    avg_risk = float(np.nanmean(risks))

    return float(dsum), avg_risk

if refined_path is not None:

    d_ref, r_ref = path_metrics(refined_path, t_fine, penalty_fine)

    baseline_d, baseline_r = path_metrics(baseline_path, t_coarse,
np.ones_like(penalty_coarse)) if baseline_path else (None, None)

    used = "refined"

elif path_coarse:

    d_ref, r_ref = path_metrics(path_coarse, t_coarse, penalty_coarse)

    baseline_d, baseline_r = path_metrics(baseline_path, t_coarse,
np.ones_like(penalty_coarse)) if baseline_path else (None, None)

    used = "coarse"

else:

    d_ref, r_ref, baseline_d, baseline_r = None, None, None, None

    used = "no_path"

results.append({

    "start": a, "end": b, "used": used,

    "riskaware_dist_m": d_ref, "riskaware_avg_penalty": r_ref,

    "baseline_dist_m": baseline_d, "baseline_avg_penalty": baseline_r

})

# save partial intermittently

try:

    pd.DataFrame(results).to_csv(partial_save, index=False)

```

except Exception:

pass

return pd.DataFrame(results)

Batch example

```
df_places = pd.read_csv("terrain_with_cognitive_fixed.csv")
```

```
places = df_places["place"].tolist()
```

```
pairs = [(places[i], places[j]) for i in range(len(places)) for j in range(i+1, len(places))]
```

```
results_df = run_experiments_batch(df_places, pairs, pad_km=8, resample_coarse=4)
```

```
results_df.to_csv("terrain_pathfinding_results_full.csv", index=False)
```

4 — Step 14: Multi-hop chained planning (memory safe)

Purpose: chain multiple segments (place sequence) planning segment-by-segment, saving results and producing an interactive HTML map. Each segment uses the coarse→refine approach, so memory usage stays bounded.

Step 14: chain_multi_hop_route

```
def chain_multi_hop_route(place_sequence, place_df, pad_km_segment=8,
resample_coarse=3, out_file="MultiHop_RiskAware_Chained.html"):
```

```
    import folium
```

```
    path_all = []
```

```
    # initialize folium map at first place
```

```
    start_row = place_df[place_df["place"]==place_sequence[0]].iloc[0]
```

```
    m = folium.Map(location=[float(start_row["lat"]), float(start_row["lon"])],
zoom_start=8)
```

```
    for i in range(len(place_sequence)-1):
```

```
        a = place_sequence[i]; b = place_sequence[i+1]
```

```
        rowA = place_df[place_df["place"]==a].iloc[0]
```

```
        rowB = place_df[place_df["place"]==b].iloc[0]
```



```

start = (float(rowA["lat"]), float(rowA["lon"]))
goal = (float(rowB["lat"]), float(rowB["lon"]))

folder = a.replace(";", "").replace(" ", "_")
slope_tif = f"terrain_data/{folder}/slope.tif"
rough_tif = f"terrain_data/{folder}/rough.tif"
if not (os.path.exists(slope_tif) and os.path.exists(rough_tif)):
    print(f"Missing rasters for segment {a}->{b}, skipping.")
    continue

penalty_coarse, t_coarse = build_fused_risk_window(slope_tif, rough_tif,
0.5*(start[0]+goal[0]), 0.5*(start[1]+goal[1]), pad_km=pad_km_segment,
resample_factor=resample_coarse)

coarse_path = astar_risk(penalty_coarse, t_coarse, start, goal)
refined_path = None
if coarse_path:
    # refine bounding box
    lats = []; lons = []
    for r,c in coarse_path:
        lonp, latp = t_coarse * (c + 0.5, r + 0.5)
        lats.append(latp); lons.append(lonp)
    center_lat = float(np.mean(lats)); center_lon = float(np.mean(lons))
    penalty_ref, t_ref = build_fused_risk_window(slope_tif, rough_tif, center_lat,
center_lon, pad_km=2, resample_factor=1)
    refined_path = astar_risk(penalty_ref, t_ref, start, goal)

chosen = refined_path if refined_path else coarse_path
if not chosen:
    print(f"No path found for {a}->{b}")

```

```

        continue

    # convert to latlon
    t_use = t_ref if refined_path else t_coarse

    seg_latlon = []

    for r,c in chosen:

        lonp, latp = t_use * (c + 0.5, r + 0.5)

        seg_latlon.append((latp, lonp))

    path_all.extend(seg_latlon)

    # draw in folium

    folium.PolyLine(seg_latlon, weight=3, opacity=0.8, tooltip=f"{a}→{b}").add_to(m)

    folium.Marker([start[0], start[1]], popup=a, icon=folium.Icon(color="green",
icon="info-sign")).add_to(m)

    folium.Marker([goal[0], goal[1]], popup=b, icon=folium.Icon(color="red",
icon="flag")).add_to(m)

    m.save(out_file)

    print("Saved chained map:", out_file)

    return out_file, path_all

```

Example

```

place_df = pd.read_csv("terrain_with_cognitive_fixed.csv")

seq = place_df["place"].tolist() # or custom ordering

chain_multi_hop_route(seq, place_df, pad_km_segment=10, resample_coarse=4)

```

5 — Step 15: Evaluation, plots & exports

Purpose: compare risk-aware vs baseline (distance-only). Save summary CSVs and plots.

Step 15: evaluation & plotting

```

import matplotlib.pyplot as plt

res = pd.read_csv("terrain_pathfinding_results_full.csv")
res["dist_ratio"] = res["riskaware_dist_m"] / res["baseline_dist_m"]
res["dist_ratio"].replace([np.inf, -np.inf], np.nan, inplace=True)
res.to_csv("pathfinding_results_summary.csv", index=False)

# Histogram: distance ratio
plt.figure(figsize=(6,4))
plt.hist(res["dist_ratio"].dropna(), bins=20)
plt.xlabel("Risk-aware distance / Baseline distance")
plt.ylabel("Count")
plt.title("Distance tradeoff: Risk-aware vs Shortest")
plt.tight_layout()
plt.savefig("dist_ratio_hist.png", dpi=200)
plt.close()

# Scatter: avg penalty vs extra distance
plt.figure(figsize=(6,4))
x = res["riskaware_avg_penalty"]
y = res["riskaware_dist_m"] - res["baseline_dist_m"]
plt.scatter(x, y)
plt.xlabel("Avg risk multiplier")
plt.ylabel("Extra distance (m) over shortest")
plt.title("Risk vs extra distance cost")
plt.tight_layout()
plt.savefig("risk_vs_extra_dist.png", dpi=200)
plt.close()

```

```
print("Saved summary CSV and plots: pathfinding_results_summary.csv,  
dist_ratio_hist.png, risk_vs_extra_dist.png")
```

Notes

- Inspect pathfinding_results_summary.csv for outliers or failed pairs.
 - If baseline_dist_m is None (baseline not found), investigate coarse grid coverage and increase pad size.
-

6 — Step 16: Final outputs, reproducibility & recommended next steps

Files produced by pipeline

- terrain_pathfinding_results_full.csv — per-pair metrics and chosen method
- pathfinding_results_summary.csv — summary and derived columns
- dist_ratio_hist.png, risk_vs_extra_dist.png — plots for manuscript
- MultiHop_RiskAware_Chained.html — interactive multi-hop folium map(s)
- batch_path_results_partial.csv — intermittently saved partial results (if used)

Reproducibility

1. Export environment (run locally):

```
conda activate smartsafe_env
```

```
conda env export --name smartsafe_env --no-builds > env_export.yml
```

2. Include cognitive_package_v2/ or instructions to acquire it; include cognitive_model.joblib if package lacks a public API.
3. Keep input folders (terrain_data/) with the exact structure used above.

Paper / Results notes (short)

- Data: SRTM DEM tiles (cite SRTM) used to compute slope & roughness.
- Terrain features: elevation-derived slope; roughness estimated from local elevation variance.
- Cognitive fusion: per-place cognitive score fused multiplicatively as penalty $1 + \alpha * \text{terrain_risk}$.
- Pathfinding: Grid-based A*; cost = distance_m × penalty; heuristic = geodesic distance.

- Evaluation: run N unique pairs; metrics: distance (m), average penalty, and distance tradeoff (risk-aware / shortest).
 - Limitations: DEM resolution, missing trails/road networks, vegetation/seasonality, simplifications of cognitive fusion.
 - Future: integrate real-time telemetry, use graph-based routing with trails/roads, include visibility/slope-based energy models.
-

7 — Troubleshooting & performance tips

- **MemoryErrors:** reduce pad_km and/or increase resample_coarse (coarser grid). Plan in segments.
 - **No path found:** ensure start & goal lie within the raster tile; try larger pad_km to include both points or merge adjacent tiles.
 - **Very long runtime:** use resample_factor > 1 (coarse), then refine around coarse path with resample_factor=1.
 - **Precision:** for final publication-quality routes, rerun final segments with resample_factor=1 over tightly cropped windows around coarse path.
 - **Large-scale experiments:** parallelize independent pair runs using multiprocessing.Pool, but ensure each process loads its window independently to avoid shared-memory contention.
-

8 — Code & parameter summary (quick copyable reference)

- build_fused_risk_window(slope_tif, rough_tif, center_lat, center_lon, pad_km=5, resample_factor=1)
- astar_risk(terrain_penalty, transform, start_latlon, goal_latlon)
- run_experiments_batch(place_rows_df, pairs, pad_km=6, resample_coarse=4)
- chain_multi_hop_route(place_sequence, place_df, pad_km_segment=8, resample_coarse=3)

Suggested defaults for experiments

- pad_km (coarse): 6–12 (depends on point separation)
- resample_coarse: 3–6 (e.g., 4)
- pad_km (refine): 2

- alpha (penalty multiplier): 2.0 (tune in sensitivity analysis)

9 — Where to place this on GitHub (recommended structure)

repo-root/

└─ code/

| └─ phase_11_16_pipeline.py # copy of code blocks above (single script)

| └─ notebooks/

| | └─ phase_11_16_runbook.ipynb # notebook with parameter cell and run sections

└─ data/

| └─ terrain_data/<PlaceFolder>/slope.tif

| └─ terrain_data/<PlaceFolder>/rough.tif

| └─ terrain_with_cognitive_fixed.csv

└─ results/

| └─ terrain_pathfinding_results_full.csv

| └─ pathfinding_results_summary.csv

| └─ dist_ratio_hist.png

| └─ risk_vs_extra_dist.png

| └─ MultiHop_RiskAware_Chained.html

└─ env_export.yml

└─ LICENSE

Important: do **not** hardcode any user name or machine-specific path. Use relative paths and parameter cells in notebooks. The code above uses relative paths (e.g., terrain_data/).
